

UNIVERSIDAD COOPERATIVA DE COLOMBIA

FACULTAD DE INGENIERÍA DE SISTEMAS (CAMPUS MONTERÍA)

ARQUITECTURA Y MODELAMIENTO DE SOFTWARE

ESTUDIANTES:

JAIME ANDRES GALEANO HERNANDEZ

RAFAEL ALEJANDRO POLO HENAO

JOHNDavid PUERTA SAEZ

LEONARDO JOSE ROMERO OVIEDO

INGENIERO:

MAURICIO RICARDO

FECHA

10/03/2026

MONTERÍA – CÓRDOBA

Estilo Arquitectónico

1. ¿Por qué microservicios?

EventHive es una plataforma digital para gestionar eventos y conferencias profesionales, actualmente está presentando problemas como:

- ✚ Caídas cuando hay muchas compras al mismo tiempo
- ✚ No escala bien cuando hay picos de tráfico
- ✚ Se generan cuellos de botella en check-ins en tiempo real

Los microservicios son fundamentales, porque cada servicio es independiente del otro, cada uno cuenta con su propia base de datos, se pueden escalar de manera individual, el despliegue puede ser independiente, se pueden aislar de mejor manera los fallos, mejoran la productividad del equipo y acortan el tiempo de implementación.

2. Ventajas frente a monolito

Monolito	Microservicios
Escala todo o nada	Escala por servicio
Solo existe una base de datos	Existen varias bases de datos independientes
Mayor acoplamiento	Desacoplamiento
Despliegue completo	Despliegue independiente
Se hace difícil el mantenimiento	Mayor modularidad

3. Posibles riesgos (latencia, complejidad, consistencia distribuida)

Mayor consumo de recursos y costos: Al tener cada servicio su propio entorno, memoria y CPU, el consumo de infraestructura suele ser mayor que en sistemas monolíticos.

Mayor complejidad: como los microservicios son distribuidos, gestionar la comunicación sobre los servicios puede resultar difícil. Puede que los desarrolladores tengan que escribir código adicional para garantizar una comunicación fluida entre los módulos.

Consistencia distribuida: el sistema puede terminar con datos contradictorios, ya que si algún servicio falla y no envía la información a las bases de datos de los otros servicios que la necesitan, genera una inconsistencia.

Principios SOLID aplicados

Single Responsibility

Cada microservicio tiene una única responsabilidad. Por ejemplo:

User Service: solo gestionara usuarios

Event Service: solo gestiona eventos

Open/Closed

Se pueden agregar nuevos servicios sin modificar los existentes. Por ejemplo:

Se puede agregar un servicio de encuestas sin modificar Ticketing

Liskov

Las diferentes implementaciones de un mismo tipo de servicio deben ser intercambiables para el cliente que los consume. Por ejemplo:

El **Notification Service** puede enviar correos vía SendGrid o AWS SES.

Interface Segregation

Cada cliente consume solo la API que necesita. Por ejemplo:

Un asistente en la App móvil solo necesita: Título, Hora y Ponente.

Dependency Inversion

Los servicios no dependen directamente entre sí, sino de contratos (REST, eventos).

Por ejemplo: Analytics depende de la "abstracción" del bus de datos (Kafka/RabbitMQ). Él escucha el evento ticket.purchased. No le importa qué base de datos usa Ticketing ni cómo procesa el pago; solo le importa el mensaje (la abstracción) de que una venta ocurrió.

Principios de Diseño Distribuido

Comunicación Asíncrona por Eventos

Los servicios se comunican mediante un intermediario (Message Broker) como Kafka o RabbitMQ.

- ✚ **Aplicación en EventHive:** Cuando el **Ticketing Service** confirma una compra, no llama al **Notification Service** directamente. Publica el evento `ticket.purchased`.
- ✚ **Por qué:** El sistema de ventas no tiene que esperar a que el correo se envíe para confirmar la compra al usuario. Mejora drásticamente el rendimiento.
- ✚ **Tolerancia a Fallos (Fault Tolerance)**
Capacidad del sistema para seguir operando aunque un componente falle.
- ✚ **Aplicación en EventHive:** Si el **Credential Service** (encargado de los QR) se cae, los usuarios aún pueden navegar por la **Agenda Service** y el **Event Service**.
- ✚ **Por qué:** En una conferencia en vivo, no puedes permitir que toda la plataforma quede inactiva solo porque el módulo de encuestas o analíticas falló.

Observabilidad Centralizada

Monitoreo total a través de logs, métricas y trazabilidad distribuida (Tracing).

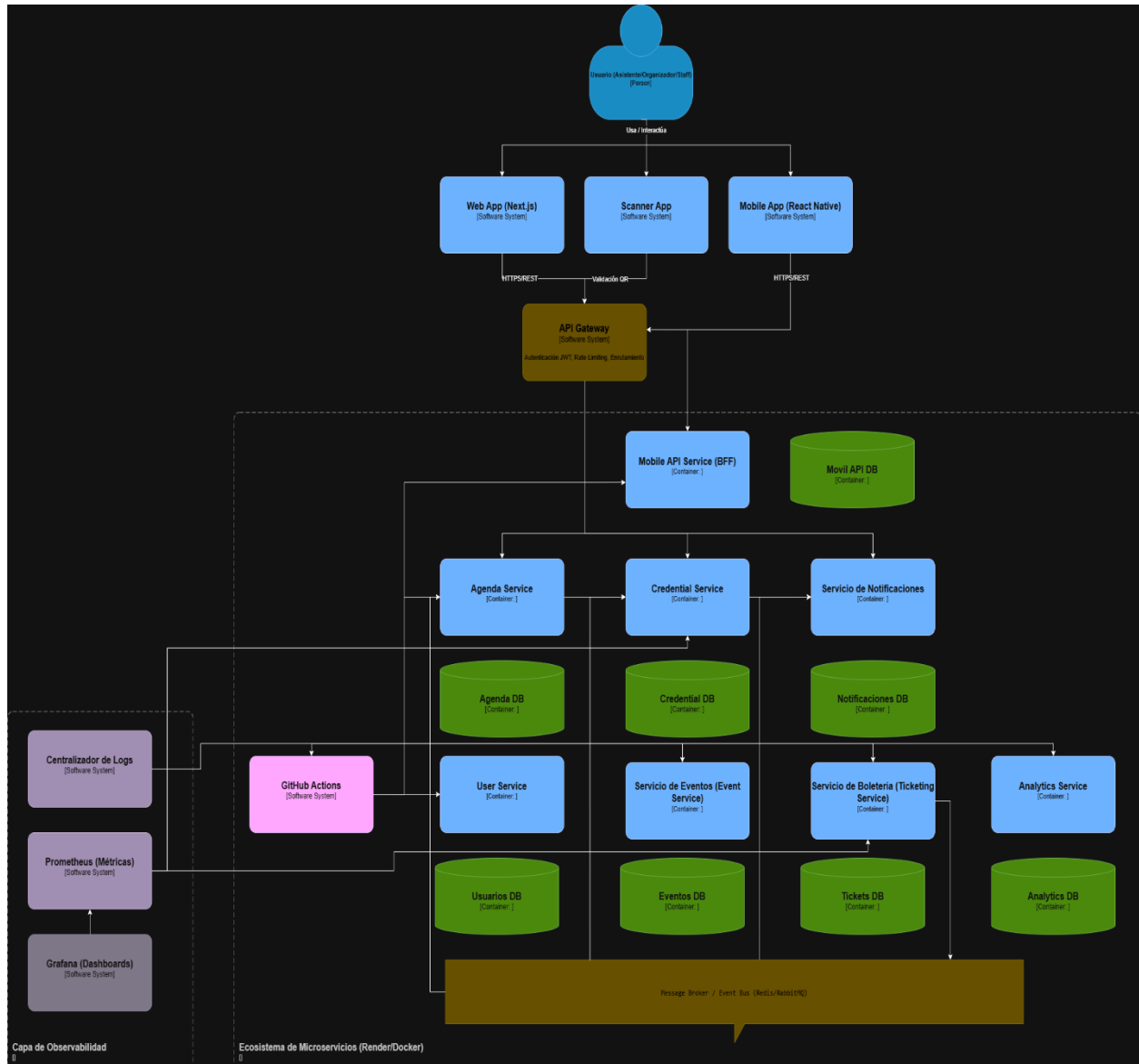
- ✚ **Aplicación en EventHive:** Uso de herramientas como
- ✚ **Prometheus** (métricas), **Grafana** y **Jaeger**. Si un usuario reporta que no recibió su QR, puedes rastrear la "huella" de esa transacción desde que entró por el **BFF** hasta que el **Notification Service** intentó enviarla.
- ✚ **Por qué:** En un sistema con 8 servicios, es imposible saber dónde ocurrió un error si no tienes un panel que unifique toda la telemetría.

Consistencia Eventual

En sistemas distribuidos, los datos no siempre están sincronizados al milisegundo en todos lados.

- ✚ **Aplicación en EventHive:** Al comprar una entrada, el **Ticketing Service** descuenta el cupo inmediatamente. Sin embargo, el **Analytics Service** podría tardar 2 segundos en reflejar la venta en el dashboard del organizador.
- ✚ **Por qué:** Intentar mantener consistencia absoluta (fuerte) en todos los servicios a la vez haría que el sistema fuera extremadamente lento e inestable.

Diagrama de Arquitectura General



Identificación y Definición de Microservicios

User Service

Responsabilidad única

Gestionar la identidad y la información de los usuarios dentro del sistema. Este servicio se encarga del registro, autenticación, administración de perfiles y control de roles de los diferentes tipos de usuarios de la plataforma, como asistentes, organizadores, ponentes y personal de staff.

Justificación de separación

La gestión de usuarios es un dominio independiente dentro del sistema y es utilizada por múltiples servicios. Separarlo como un microservicio permite centralizar la autenticación y autorización, facilitar la reutilización de la información de usuarios y mejorar la seguridad del sistema. Además, permite escalar y mantener el sistema de identidad de manera independiente de otras funcionalidades como eventos o boletos.

Lenguaje

JavaScript / TypeScript con Node.js.

Endpoints principales

- ✚ Registro de usuario
- ✚ POST /users/register
- ✚ Inicio de sesión
- ✚ POST /users/login
- ✚ Consultar usuario
- ✚ GET /users/{id}
- ✚ Actualizar perfil
- ✚ PUT /users/{id}
- ✚ Listar usuarios
- ✚ GET /users

Base de datos asociada

Usuarios DB

Tipo de base de datos: PostgreSQL.

Tablas principales:

- + users
- + roles
- + user_roles

Datos almacenados:

- + id_usuario
- + nombre
- + email
- + contraseña encriptada
- + rol
- + fecha de registro

Tipo de comunicación con otros servicios

Comunicación síncrona mediante REST API con:

- + Event Service
- + Ticketing Service
- + Credential Service

Para validar información del usuario o asociar usuarios con eventos y boletos.

También puede participar en comunicación asíncrona mediante eventos a través del broker de mensajería del sistema.

Eventos que publica o consume

- + **user.created**
- + **user.updated**
- + **user.deleted**

Event Service

Responsabilidad única

Gestionar toda la información relacionada con los eventos dentro de la plataforma. Este microservicio se encarga de la creación, actualización, consulta y administración de eventos organizados en el sistema.

El servicio permite a los organizadores registrar eventos, modificar su información y consultar los detalles asociados como fecha, ubicación, capacidad y descripción.

Justificación de separación

La gestión de eventos constituye uno de los dominios principales del sistema. Separar esta funcionalidad en un microservicio independiente permite aislar la lógica relacionada con la organización y administración de eventos, facilitando su escalabilidad y mantenimiento.

Además, varios servicios del sistema dependen de la información de eventos, como el **Ticketing Service**, **Agenda Service** y **Analytics Service**, por lo que mantener esta funcionalidad desacoplada mejora la modularidad y permite que cada servicio evolucione de forma independiente.

Lenguaje

JavaScript / TypeScript con Node.js.

Endpoints principales

-  Crear evento
-  POST /events
-  Consultar evento
-  GET /events/{id}
-  Listar eventos
-  GET /events
-  Actualizar evento
-  PUT /events/{id}
-  Eliminar evento
-  DELETE /events/{id}
-  Consultar eventos por organizador
-  GET /events/organizer/{organizerId}

Base de datos asociada

Eventos DB

Tipo de base de datos: PostgreSQL.

Tablas principales:

- + events
- + event_organizers
- + event_locations

Datos almacenados incluyen:

- + id_evento
- + nombre_evento
- + descripción
- + fecha_inicio
- + fecha_fin
- + ubicación
- + capacidad
- + id_organizador

Tipo de comunicación con otros servicios

Comunicación síncrona mediante REST API con:

- + **User Service** (para validar organizadores)
- + **Agenda Service** (para asociar sesiones o actividades)
- + **Ticketing Service** (para gestionar venta de boletos del evento)

También puede utilizar **comunicación asíncrona mediante eventos** a través del broker de mensajería del sistema para notificar cambios en los eventos.

Eventos que publica:

- + event.created
- + event.updated
- + event.cancelled

Eventos que consume:

- + user.deleted

Estos eventos permiten que otros servicios del sistema se mantengan sincronizados cuando se crean, modifican o cancelan eventos.

Ticketing Service

Responsabilidad única

Gestionar la venta, reserva y administración de boletos para los eventos dentro de la plataforma. Este servicio se encarga de procesar la compra de entradas, verificar la disponibilidad de boletos y registrar las transacciones asociadas a cada evento.

También mantiene el registro de los tickets adquiridos por los usuarios y controla el número de entradas disponibles para cada evento.

Justificación de separación

La gestión de boletos representa un dominio crítico dentro del sistema, ya que involucra procesos de compra, control de disponibilidad y registro de transacciones. Separar esta funcionalidad en un microservicio independiente permite manejar de forma segura y escalable las operaciones relacionadas con la venta de entradas.

Además, este servicio interactúa con varios otros servicios como Event Service, Credential Service, Notification Service y Analytics Service, por lo que mantenerlo desacoplado facilita la integración mediante eventos y mejora la resiliencia del sistema.

Lenguaje

JavaScript / TypeScript con Node.js.

Endpoints principales

-  Comprar ticket
-  POST /tickets/purchase

-  Consultar ticket
-  GET /tickets/{id}

-  Consultar tickets de un usuario
-  GET /tickets/user/{userId}

-  Consultar tickets de un evento
-  GET /tickets/event/{eventId}

-  Cancelar ticket
-  DELETE /tickets/{id}

Base de datos asociada

Tipo de comunicación con otros servicios

Comunicación síncrona mediante REST API con:

- + Event Service (para verificar la existencia del evento y su capacidad)
- + User Service (para validar el usuario que realiza la compra)

Comunicación asíncrona mediante eventos con otros servicios a través del broker de mensajería.

Eventos que publica o consume

Eventos que publica:

- + ticket.purchased
- + ticket.cancelled
- + ticket.refunded

Eventos que consume:

event.cancelled

Cuando un ticket es comprado, el evento ticket.purchased puede ser utilizado por:

- + Credential Service para generar la credencial digital o código QR
- + Notification Service para enviar confirmación al usuario
- + Analytics Service para actualizar métricas del sistema

Credential Service

Responsabilidad única

Gestionar la generación y validación de credenciales digitales para los asistentes a los eventos. Este servicio se encarga de crear credenciales únicas para cada ticket comprado, normalmente en forma de códigos QR o identificadores digitales que permiten el acceso al evento.

Además, valida estas credenciales cuando los asistentes ingresan al evento utilizando aplicaciones de escaneo.

Justificación de separación

La gestión de credenciales y control de acceso es una funcionalidad crítica del sistema que requiere seguridad y validación rápida. Separar esta responsabilidad en un microservicio independiente permite manejar de forma eficiente la generación y verificación de credenciales sin afectar otros procesos del sistema.

Además, este servicio debe interactuar con dispositivos de escaneo en tiempo real durante los eventos, por lo que mantenerlo desacoplado facilita su escalabilidad y disponibilidad.

Lenguaje

JavaScript / TypeScript con Node.js.

Endpoints principales

- ✚ Generar credencial
- ✚ POST /credentials
- ✚ Consultar credencial
- ✚ GET /credentials/{id}
- ✚ Validar credencial (escaneo QR)
- ✚ POST /credentials/validate
- ✚ Consultar credenciales de un evento
- ✚ GET /credentials/event/{eventId}

Base de datos asociada

- ✚ Credentials DB
- ✚ Tipo de base de datos: PostgreSQL.

Tablas principales:

- ✚ credentials
- ✚ credential_logs

Datos almacenados incluyen:

- + id_credencial
- + id_ticket
- + id_evento
- + código_QR
- + estado
- + fecha_generación
- + fecha_validación

Tipo de comunicación con otros servicios

Comunicación síncrona mediante REST API con:

- + Ticketing Service para verificar que el ticket es válido
- + Event Service para validar el evento asociado

Comunicación asíncrona mediante eventos con otros servicios del sistema.

Eventos que publica o consume

Eventos que publica:

- + credential.generated
- + credential.validated

Eventos que consume:

- + ticket.purchased
- + ticket.cancelled

Cuando ocurre el evento **ticket.purchased**, el servicio genera automáticamente la credencial digital del usuario.

Cuando una credencial es escaneada en la entrada del evento, se publica el evento **credential.validated**, el cual puede ser utilizado por el **Analytics Service** para registrar asistencia.

Agenda Service

Responsabilidad única

Gestionar la agenda o programación de actividades dentro de cada evento. Este microservicio permite crear, modificar y consultar las sesiones, conferencias, talleres o actividades que forman parte del cronograma de un evento.

También permite organizar el horario, los ponentes y la ubicación de cada actividad dentro del evento.

Justificación de separación

La gestión de la agenda representa un dominio funcional específico dentro del sistema de eventos. Separar esta funcionalidad en un microservicio independiente permite manejar la programación de actividades de forma modular y escalable.

Además, los eventos pueden tener múltiples sesiones y cambios frecuentes en el cronograma, por lo que mantener esta lógica desacoplada facilita la actualización de la agenda sin afectar otros servicios como la gestión de boletos o usuarios.

Lenguaje

JavaScript / TypeScript con Node.js.

Endpoints principales

-  Crear sesión o actividad
-  POST /agenda
-  Consultar sesiones de un evento
-  GET /agenda/event/{eventId}
-  Consultar una sesión específica
-  GET /agenda/{sessionId}
-  Actualizar sesión
-  PUT /agenda/{sessionId}
-  Eliminar sesión
-  DELETE /agenda/{sessionId}

Base de datos asociada

Agenda DB

Tipo de base de datos: PostgreSQL.

Tablas principales:

- + sessions
- + speakers
- + session_locations

Datos almacenados incluyen:

- + id_session
- + id_evento
- + título_sesión
- + descripción
- + hora_inicio
- + hora_fin
- + ponente
- + ubicación

Tipo de comunicación con otros servicios

Comunicación síncrona mediante REST API con:

- + Event Service para verificar que el evento exista
- + User Service para validar los ponentes o moderadores

Comunicación asíncrona mediante eventos con otros servicios del sistema.

Eventos que publica o consume

Eventos que publica:

- + session.created
- + session.updated
- + session.deleted

Eventos que consume:

event.cancelled

Cuando un evento es cancelado, el Agenda Service elimina o desactiva automáticamente todas las sesiones asociadas.

Notification Service

Responsabilidad única

Gestionar el envío de notificaciones a los usuarios del sistema. Este servicio se encarga de enviar mensajes informativos relacionados con las acciones realizadas dentro de la plataforma, como confirmaciones de compra de tickets, recordatorios de eventos, actualizaciones en la agenda o cambios importantes en un evento.

Las notificaciones pueden enviarse mediante correo electrónico, notificaciones push o mensajes dentro de la plataforma.

Justificación de separación

El envío de notificaciones es una funcionalidad transversal que puede ser utilizada por múltiples servicios del sistema. Separarla como un microservicio independiente permite centralizar la gestión de mensajes, mejorar la escalabilidad del sistema y evitar que otros servicios se encarguen directamente del envío de comunicaciones.

Además, este servicio funciona principalmente mediante eventos asíncronos, reaccionando a acciones generadas por otros microservicios como la compra de tickets o la creación de eventos.

Lenguaje

JavaScript / TypeScript con Node.js.

Endpoints principales

- ✚ Enviar notificación
- ✚ POST /notifications
- ✚ Consultar notificaciones de un usuario
- ✚ GET /notifications/user/{userId}
- ✚ Marcar notificación como leída
- ✚ PUT /notifications/{id}/read

Base de datos asociada

Notifications DB

Tipo de base de datos: MongoDB.

+ Tablas / colecciones principales:

+ notifications

+ notification_logs

+ Datos almacenados incluyen:

+ id_notificación

+ id_usuario

+ tipo_notificación

+ mensaje

+ estado (enviada, leída)

+ fecha_envío

Tipo de comunicación con otros servicios

Principalmente comunicación asíncrona mediante eventos provenientes de otros microservicios del sistema.

También puede utilizar REST API con User Service para obtener información de contacto del usuario.

Eventos que publica o consume

Eventos que consume:

+ ticket.purchased

+ event.created

+ session.updated

+ credential.generated

Cuando recibe alguno de estos eventos, el servicio genera y envía la notificación correspondiente al usuario.

Eventos que publica:

+ notification.sent

Este evento puede ser utilizado por Analytics Service para medir la interacción de los usuarios con el sistema.

Analytics Service

Responsabilidad única

Recolectar, procesar y analizar datos generados por los diferentes microservicios del sistema con el objetivo de generar estadísticas y reportes sobre el funcionamiento de la plataforma y el desempeño de los eventos.

Este servicio permite a los organizadores obtener información como número de tickets vendidos, asistencia real a los eventos, popularidad de sesiones y comportamiento de los usuarios dentro de la plataforma.

Justificación de separación

El análisis de datos requiere procesar información proveniente de múltiples servicios del sistema, como la compra de tickets, validación de credenciales y actividades de los eventos. Separar esta funcionalidad en un microservicio independiente permite centralizar el procesamiento de datos sin afectar el rendimiento de los servicios operacionales.

Además, este servicio puede escalar de manera independiente y utilizar tecnologías especializadas para análisis y visualización de datos.

Lenguaje

Python o Node.js.

Endpoints principales

-  Consultar estadísticas de un evento
 GET /analytics/event/{eventId}
-  Consultar métricas de ventas
 GET /analytics/tickets/{eventId}
-  Consultar estadísticas de asistencia
 GET /analytics/attendance/{eventId}
-  Generar reporte general
 GET /analytics/report/{eventId}

Base de datos asociada

Analytics DB

Tipo de base de datos: MongoDB o Data Warehouse.

Colecciones principales:

- + event_metrics
- + ticket_sales
- + attendance_records
- + user_activity

Datos almacenados incluyen:

- + id_evento
- + tickets_vendidos
- + tickets_validados
- + sesiones_más_asistidas
- + tasa_de_asistencia
- + actividad_de_usuarios

Tipo de comunicación con otros servicios

Principalmente comunicación asíncrona basada en eventos, recibiendo información generada por otros microservicios.

También puede utilizar REST API para que los organizadores consulten reportes y estadísticas.

Eventos que publica o consume

Eventos que consume:

- + ticket.purchased
- + ticket.cancelled
- + credential.validated
- + session.created
- + notification.sent

Eventos que publica:

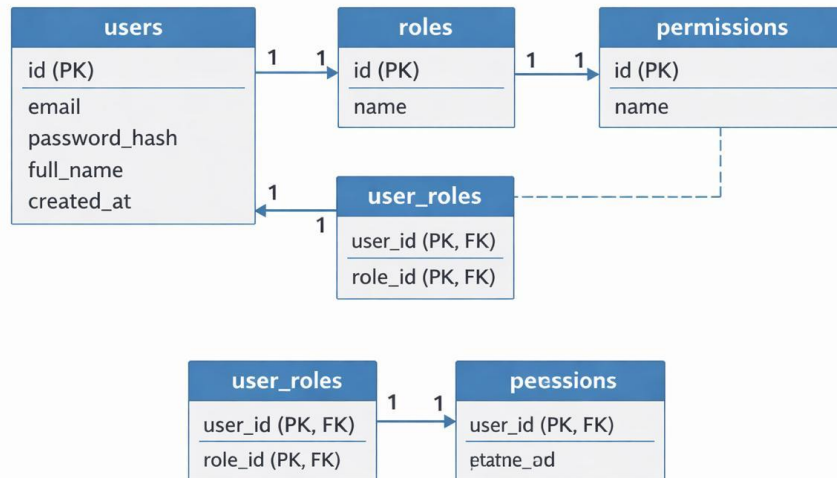
- + analytics.report.generated

Estos eventos permiten generar métricas que ayuden a los organizadores a tomar decisiones basadas en datos.

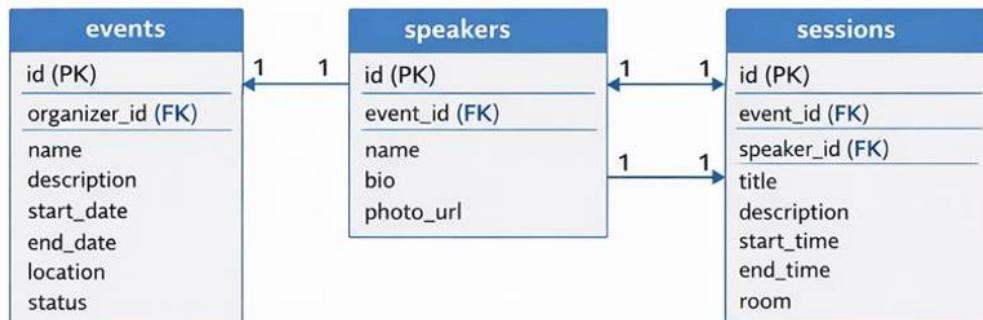
DISEÑO DEL MODELO DE DATOS RELACIONAL PARA CADA MICROSERVICIO:

ENTIDAD – RELACION

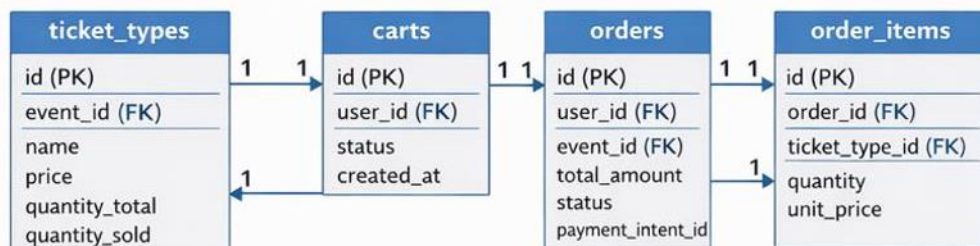
User Service



Event Service



Ticketing Service





Credential Service

credentials
id (PK)
order_item_id (FK)
ticket_type_id (FK)
attendee_name
qr_code_hash
is_used
used_at
used_by

Analytics Service

daily_sales
id (PK)
event_id
sale_date
total_revenue

1

session_attendance
id (PK)
session_id
event_id (FK)
attendees_count
checkin_time

N

attendee_demographics
id (PK)
event_id (FK)
location
age_group
ticket_type

TABLAS USERS

users

Campo	Tipo	Descripción
id	UUID	Identificador del usuario
email	VARCHAR	Correo del usuario
password_hash	TEXT	Contraseña cifrada
full_name	VARCHAR	Nombre completo
created_at	TIMESTAMP	Fecha de registro

roles

Campo	Tipo
id	UUID
name	VARCHAR

permisos

Campo	Tipo
id	UUID
name	VARCHAR

user_roles

Campo	Tipo
user_id	UUID
role_id	UUID

TABLAS EVENTS

EVENTS

Campo	Tipo
id	UUID
organizer_id	UUID
name	VARCHAR
description	TEXT
start_date	DATE
end_date	DATE
location	VARCHAR
status	VARCHAR

SPEAKERS

Campo	Tipo
id	UUID
event_id	UUID
name	VARCHAR
bio	TEXT
photo_url	TEXT

SESSIONS

Campo	Tipo
id	UUID
event_id	UUID
speaker_id	UUID
title	VARCHAR
description	TEXT
start_time	TIMESTAMP
end_time	TIMESTAMP
room	VARCHAR

TABLAS TICKETING SERVICE

TICKET_TYPES

Campo	Tipo
id	UUID
event_id	UUID
name	VARCHAR
price	DECIMAL
quantity_total	INT
quantity_sold	INT

ORDERS

Campo	Tipo
id	UUID
user_id	UUID
event_id	UUID
total_amount	DECIMAL
status	VARCHAR
payment_intent_id	VARCHAR

ORDES_ITEMS

Campo	Tipo
id	UUID
order_id	UUID
ticket_type_id	UUID
quantity	INT
unit_price	DECIMAL

CARTS

Campo	Tipo
id	UUID
user_id	UUID
status	VARCHAR
created_at	TIMESTAMP

LLAVES PRIMARIAS PARA CADA MICROSERVICIOS

USER

- ✚ users.id
- ✚ roles.id
- ✚ permissions.id
- ✚ user_roles (user_id, role_id)

EVENTS

- ✚ events.id
- ✚ speakers.id
- ✚ sessions.id

TICKETING SERVICE

- ✚ ticket_types.id
- ✚ carts.id
- ✚ orders.id
- ✚ order_items.id
- ✚ order_items.order_id → orders.id
- ✚ order_items.ticket_type_id → ticket_types.id

LLAVES FORANEAS PARA CADA MICROSERVICIOS

USER

- ✚ user_roles.user_id → users.id
- ✚ user_roles.role_id → roles.id

EVENTS

- ✚ speakers.event_id → events.id
- ✚ sessions.event_id → events.id
- ✚ sessions.speaker_id → speakers.id

TICKETING SERVICE

- ✚ índice en ticket_types.event_id
- ✚ índice en orders.status

INDICES PARA CADA MICROSERVICIOS

USER

- ✚ índice en users.email
- ✚ índice en user_roles.user_id

EVENTS

- ✚ índice en events.organizer_id
- ✚ índice en sessions.event_id

TICKETING SERVICE

- ✚ índice en ticket_types.event_id
- ✚ índice en orders.status

RESTRICCIONES PARA CADA MICROSERVICIOS

USER

- ✚ email debe ser único
- ✚ password_hash no puede ser nulo
- ✚ role_id no puede ser nulo

EVENTS

- ✚ start_date debe ser menor que end_date
- ✚ title no puede ser nulo

TICKETING SERVICE

- ✚ quantity debe ser mayor que 0
- ✚ quantity_total debe ser mayor o igual a quantity_sold

TIPO DE NORMALIZACION PARA CADA MICROSERVICIOS

USER

El modelo se encuentra en **Tercera Forma Normal (3NF)** debido a que:

- ✚ cada atributo depende únicamente de la clave primaria
- ✚ no existen dependencias transitivas
- ✚ se evita redundancia en los datos

EVENTS

El modelo se encuentra en **Tercera Forma Normal (3NF)**.

TICKETING SERVICE

El modelo se encuentra en **3NF**, evitando redundancia y facilitando la gestión de múltiples entradas por orden.

JUSTIFICACIÓN DEL DISEÑO USER SERVICE

El User Service es responsable de gestionar la identidad de los usuarios dentro de la plataforma EventHive, incluyendo asistentes, organizadores y personal del evento. Este microservicio se diseñó de forma independiente para centralizar la autenticación, autorización y gestión de perfiles.

El modelo de datos se compone principalmente de las tablas users, roles, permissions y user_roles, lo que permite implementar un sistema de control de acceso basado en roles (RBAC). Este enfoque facilita la asignación de diferentes niveles de acceso según el tipo de usuario y sus responsabilidades dentro de la plataforma.

Separar esta funcionalidad en un microservicio independiente permite escalar el sistema de autenticación sin afectar al resto de los servicios, además de mantener una gestión centralizada de la seguridad del sistema.

INTEGRIDAD SIN JOINS ENTRE SERVICIOS

En una arquitectura de microservicios, cada servicio posee su propia base de datos. Por esta razón, los demás microservicios no realizan consultas directas ni joins a la base de datos del User Service.

En su lugar, utilizan identificadores únicos como user_id, los cuales se transmiten a través de tokens de autenticación JWT. Cada servicio puede validar la identidad del usuario mediante el token sin necesidad de acceder directamente a la base de datos del servicio de usuarios.

ESTRATEGIA DE CONSISTENCIA EVENTUAL

La consistencia eventual se mantiene mediante un sistema de eventos. Cuando ocurre un cambio relevante en la información del usuario, como la creación o actualización de un perfil, el User Service publica eventos como:

- 🚦 user.created
- 🚦 user.updated

Otros microservicios pueden suscribirse a estos eventos para actualizar sus propios datos o realizar acciones adicionales cuando sea necesario.

JUSTIFICACIÓN DEL DISEÑO EVENT SERVICE

El Event Service es responsable de gestionar toda la información relacionada con los eventos organizados en la plataforma. Esto incluye la creación de eventos, configuración de agendas, sesiones, ponentes y detalles logísticos.

El modelo de datos se organiza en entidades como events, speakers y sessions, lo que permite representar de forma clara la estructura jerárquica de un evento. Cada evento puede contener múltiples sesiones y cada sesión puede estar asociada a uno o varios ponentes.

Este diseño facilita la gestión de eventos complejos con múltiples actividades y permite que el servicio sea consultado por aplicaciones web y móviles sin depender de otros servicios.

INTEGRIDAD SIN JOINS ENTRE SERVICIOS

Otros microservicios como Ticketing Service o Analytics Service no realizan joins directos con la base de datos del Event Service. En su lugar, utilizan identificadores como event_id o session_id para referenciar eventos o sesiones.

Cuando es necesario obtener información adicional, los servicios realizan consultas a través de la API REST del Event Service, manteniendo así la independencia de las bases de datos.

ESTRATEGIA DE CONSISTENCIA EVENTUAL

Cuando se producen cambios en la configuración de un evento, el servicio publica eventos como:

- ✚ event.created
- ✚ event.updated
- ✚ session.created

Estos eventos permiten que otros microservicios actualicen su información de forma asincrónica y mantengan consistencia eventual.

JUSTIFICACIÓN DEL DISEÑO TICKETING SERVICE

El Ticketing Service es el componente encargado de gestionar la venta de entradas para los eventos. Debido a la posibilidad de picos de tráfico durante la venta de entradas, este servicio debe ser altamente escalable y capaz de manejar múltiples transacciones simultáneamente.

El modelo de datos incluye las tablas `ticket_types`, `carts`, `cart_items`, `orders` y `order_items`. Esta estructura permite separar claramente el catálogo de entradas, el carrito de compra y los órdenes de pago, facilitando la gestión del proceso de compra.

Este diseño también permite garantizar la integridad transaccional y evitar problemas como la sobreventa de entradas.

INTEGRIDAD SIN JOINS ENTRE SERVICIOS

El Ticketing Service no realiza joins con otras bases de datos. En su lugar, utiliza identificadores como `user_id` y `event_id` para relacionar las órdenes con los usuarios y los eventos correspondientes.

Cuando es necesario validar información externa, el servicio consulta las APIs de otros microservicios mediante solicitudes REST.

ESTRATEGIA DE CONSISTENCIA EVENTUAL

Una vez que una compra se completa exitosamente, el Ticketing Service publica el evento:

`ticket.purchased`

Este evento es consumido por varios servicios del sistema, como Credential Service, Notification Service, Event Service y Analytics Service. Gracias a este mecanismo, cada servicio puede reaccionar de forma independiente sin necesidad de comunicación directa.

JUSTIFICACIÓN DEL DISEÑO CREDENTIAL SERVICE

El Credential Service es responsable de generar y validar las credenciales digitales de acceso al evento, principalmente en forma de códigos QR.

El modelo de datos se centra en la tabla credentials, que almacena información como el identificador de la orden, el tipo de entrada, el hash del código QR y el estado de uso del mismo.

El uso de un hash criptográfico del código QR permite realizar validaciones rápidas y seguras en los puntos de acceso, evitando falsificaciones o reutilización indebida de credenciales.

INTEGRIDAD SIN JOINS ENTRE SERVICIOS

Este servicio no realiza consultas directas a la base de datos del Ticketing Service. En lugar de ello, recibe la información necesaria mediante eventos generados durante el proceso de compra.

El identificador order_item_id permite relacionar cada credencial con una entrada específica sin necesidad de realizar joins entre bases de datos.

ESTRATEGIA DE CONSISTENCIA EVENTUAL

El flujo de generación de credenciales funciona mediante eventos asincrónicos:

- 1.El Ticketing Service publica el evento ticket.purchased.
- 2.El Credential Service consume este evento y genera el código QR correspondiente.
- 3.Posteriormente publica el evento qr.generated, el cual puede ser utilizado por otros servicios como Notification Service para enviar la credencial al usuario.

JUSTIFICACIÓN DEL DISEÑO ANALYTICS SERVICE

El Analytics Service se encarga de procesar y almacenar métricas analíticas relacionadas con ventas, asistencia a sesiones y perfil demográfico de los asistentes.

Las tablas principales incluyen daily_sales, session_attendance y attendee_demographics, las cuales almacenan datos agregados que permiten generar dashboards para los organizadores del evento.

Separar esta funcionalidad en un servicio independiente permite ejecutar consultas analíticas complejas sin afectar el rendimiento de los servicios transaccionales del sistema.

INTEGRIDAD SIN JOINS ENTRE SERVICIOS

El Analytics Service no realiza consultas directas a las bases de datos de otros servicios. En su lugar, obtiene la información mediante eventos publicados por los microservicios del sistema.

De esta manera, se mantiene la independencia de las bases de datos y se evita el acoplamiento entre servicios.

ESTRATEGIA DE CONSISTENCIA EVENTUAL

La información analítica se actualiza mediante la suscripción a eventos como:

- ✚ ticket.purchased
- ✚ checkin.processed
- ✚ session.rated

Estos eventos alimentan procesos ETL que transforman y agregan los datos en las tablas analíticas. Aunque los datos pueden tardar algunos segundos o minutos en reflejarse en los dashboards, el sistema mantiene consistencia eventual sin afectar el rendimiento de las operaciones principales.

1. REST SÍNCRONO

La comunicación REST síncrona se utiliza cuando un servicio necesita una respuesta inmediata para continuar una operación o devolver un resultado al cliente.

Casos de uso en EventHive

- ✚ El API Gateway consulta al User Service para validar autenticación y permisos.
- ✚ El Ticketing Service consulta al Event Service para verificar que un evento exista, esté publicado y tenga ventas habilitadas.
- ✚ La aplicación móvil o web consulta al Agenda Service para obtener sesiones, horarios y favoritos.
- ✚ Los dispositivos de acceso consultan al Credential Service para validar un QR en tiempo real.
- ✚ El Mobile API Service agrega respuestas de varios servicios para entregarlas en un formato optimizado a la app móvil.

Ventajas

- ✚ Respuesta inmediata.
- ✚ Simplicidad de implementación.
- ✚ Adecuado para validaciones previas y consultas de lectura.

Desventajas

- ✚ Acoplamiento temporal entre servicios.
- ✚ Mayor riesgo de latencia acumulada.
- ✚ Propagación de fallos si un servicio dependiente no responde.

En EventHive, REST se reserva para:

- ✚ autenticación y autorización,
- ✚ validaciones de existencia o estado,
- ✚ consultas de lectura inmediata,
- ✚ operaciones críticas de baja tolerancia a espera, como validación de QR.

2. EVENTOS ASÍNCRONOS

La comunicación asíncrona se usa para desacoplar procesos que no necesitan terminar dentro de la misma solicitud del usuario. Para ello, los servicios publican y consumen eventos de dominio a través de un broker, preferiblemente RabbitMQ por ofrecer mejores garantías de entrega que un Pub/Sub simple.

Casos de uso en EventHive

Cuando el Ticketing Service confirma una compra, publica el evento `ticket.purchased`. A partir de ese momento:

- ✚ el Credential Service consume el evento y genera la credencial o QR;
- ✚ el Notification Service consume el resultado y envía el email o push;
- ✚ el Analytics Service consume la compra para actualizar métricas;
- ✚ el Event Service puede actualizar una proyección local de aforo vendido.

De forma similar, cuando el Credential Service registra un ingreso, publica `checkin.processed`, que puede ser consumido por Analytics y por paneles de monitoreo.

Ventajas

- ✚ Bajo acoplamiento.
- ✚ Mejor escalabilidad.
- ✚ Mayor resiliencia.
- ✚ Permite procesamiento en paralelo.

Desventajas

- ✚ No existe consistencia inmediata entre todos los servicios.
- ✚ Mayor complejidad operativa.
- ✚ Requiere idempotencia y monitoreo de mensajes.
- ✚ Eventos sugeridos
 - ✚ ticket.purchased
 - ✚ payment.failed
 - ✚ qr.generated
 - ✚ checkin.processed
 - ✚ session.reminder.requested
 - ✚ event.updated
 - ✚ order.cancelled

3. PATRÓN SAGA

Sí aplica, especialmente en el flujo de compra de entradas, donde intervienen varias acciones distribuidas y no es viable usar una única transacción ACID entre microservicios.

- ✚ Flujo de Saga propuesto
- ✚ El Ticketing Service crea una orden en estado PENDING.
- ✚ Reserva inventario de entradas.
- ✚ Solicita el cobro a la pasarela de pago.
- ✚ Si el pago es exitoso, cambia la orden a PAID.
- ✚ Publica el evento ticket.purchased.
- ✚ El Credential Service genera la credencial.
- ✚ El Notification Service envía la confirmación.
- ✚ Compensaciones
- ✚ Si el pago falla:
 - ✚ la orden cambia a PAYMENT_FAILED,
 - ✚ se libera el inventario reservado,
 - ✚ se publica un evento de fallo o liberación.
- ✚ Si la generación de QR falla:
 - ✚ la orden puede permanecer como PAID,
 - ✚ el sistema reintenta la generación,
 - ✚ si el problema persiste, se envía a una cola de errores y se marca para intervención operativa.

Tipo de Saga recomendado

Para EventHive conviene una Saga coreografiada, donde cada servicio reacciona a eventos sin un coordinador central. Esto encaja bien porque el flujo posterior a la compra es naturalmente desacoplado.

No obstante, si en el futuro el flujo de compra incorpora más reglas y decisiones, podría migrarse a una Saga orquestada para centralizar la lógica.

4. GESTIÓN DE ERRORES

- ✚ La estrategia de errores debe distinguir entre errores funcionales y errores técnicos.

- ✚ Errores funcionales

- ✚ Son errores esperables de negocio:

- ✚ entradas agotadas,

- ✚ cupón inválido,

- ✚ evento no publicado,

- ✚ QR ya utilizado,

- ✚ sesión fuera de horario.

- ✚ Estos errores no deben reintentarse automáticamente. Deben devolverse con mensajes claros y códigos controlados.

- ✚ Errores técnicos

- ✚ Son fallos transitorios o de infraestructura:

- ✚ timeout,

- ✚ caída temporal de un servicio,

- ✚ error de red,

- ✚ indisponibilidad del broker,

- ✚ error temporal de la pasarela de pago.

Manejo en REST

Las APIs deben responder con códigos HTTP consistentes:

- ✚ 200/201 para éxito,
- ✚ 400 para error funcional de validación,
- ✚ 401/403 para autenticación/autorización,
- ✚ 404 para recurso inexistente,
- ✚ 409 para conflictos como sobreventa o estado inválido,
- ✚ 500/502/503/504 para fallos técnicos.

También se recomienda un formato estándar de error:

```
{  
  "error": "TICKET_SOLD_OUT",  
  "message": "No hay cupos disponibles para este tipo de entrada",  
  "trace_id": "abc-123"  
}
```

Manejo en eventos

En comunicación asíncrona:

- ✚ el consumidor debe ser idempotente;
- ✚ si el procesamiento falla, se reintenta;
- ✚ si supera el número máximo de intentos, el mensaje se mueve a una Dead Letter Queue (DLQ);

todos los mensajes deben llevar event_id y correlation_id para trazabilidad.

5. TIMEOUTS

- ✚ Los timeouts evitan que una solicitud quede bloqueada indefinidamente esperando una dependencia.
- ✚ Propuesta conceptual para EventHive
- ✚ Validación interna de usuario o evento: 300 ms a 800 ms
- ✚ Consulta agregada del BFF móvil: 1 a 2 segundos
- ✚ Validación de QR en accesos: 100 ms a 300 ms
- ✚ Llamadas a pasarela de pago: 3 a 5 segundos
- ✚ Publicación o consumo de eventos: controlado por el broker y por tiempos máximos de procesamiento

Criterio

Los servicios más críticos en tiempo real, como validación de credenciales, deben usar timeouts más estrictos.

Las dependencias externas, como pasarelas de pago, requieren un margen mayor, pero siempre controlado.

6. RETRIES

Los reintentos se aplican solo cuando el error es potencialmente transitorio.

Casos donde sí aplicar retries: timeout de red, desconexión momentánea, error temporal del proveedor de correo, fallo temporal del broker, respuesta 502/503/504 de una dependencia.

Casos donde no aplicar retries

entrada agotada, orden inválida, QR ya usado, cupón vencido, permiso insuficiente.

Estrategia recomendada

backoff exponencial, agregar jitter para evitar picos sincronizados, máximo de 3 a 5 intentos, después enviar a DLQ o marcar incidencia.

Ejemplo conceptual

Si el Notification Service no puede enviar un correo porque el proveedor SMTP está caído, reintenta varias veces con espera progresiva. Si todos los intentos fallan, registra el fallo y deja el mensaje en cola de errores para revisión.

7. CIRCUIT BREAKER CONCEPTUAL

- ✚ El circuit breaker protege al sistema frente a dependencias inestables, evitando insistir sobre servicios que están fallando y previniendo un efecto cascada.
- ✚ Dónde aplicarlo en EventHive
 - ✚ entre Ticketing Service y la pasarela de pago;
 - ✚ entre Mobile API Service y servicios internos agregados;
 - ✚ entre Notification Service y proveedores de email/push;
 - ✚ opcionalmente entre el API Gateway y servicios con historial de latencia variable.
- ✚ Estados del circuit breaker
 - ✚ Closed: las llamadas fluyen normalmente.
 - ✚ Open: se bloquean temporalmente las llamadas porque la dependencia presenta demasiados fallos.
 - ✚ Half-open: se permite un pequeño número de pruebas para verificar si la dependencia se recuperó.

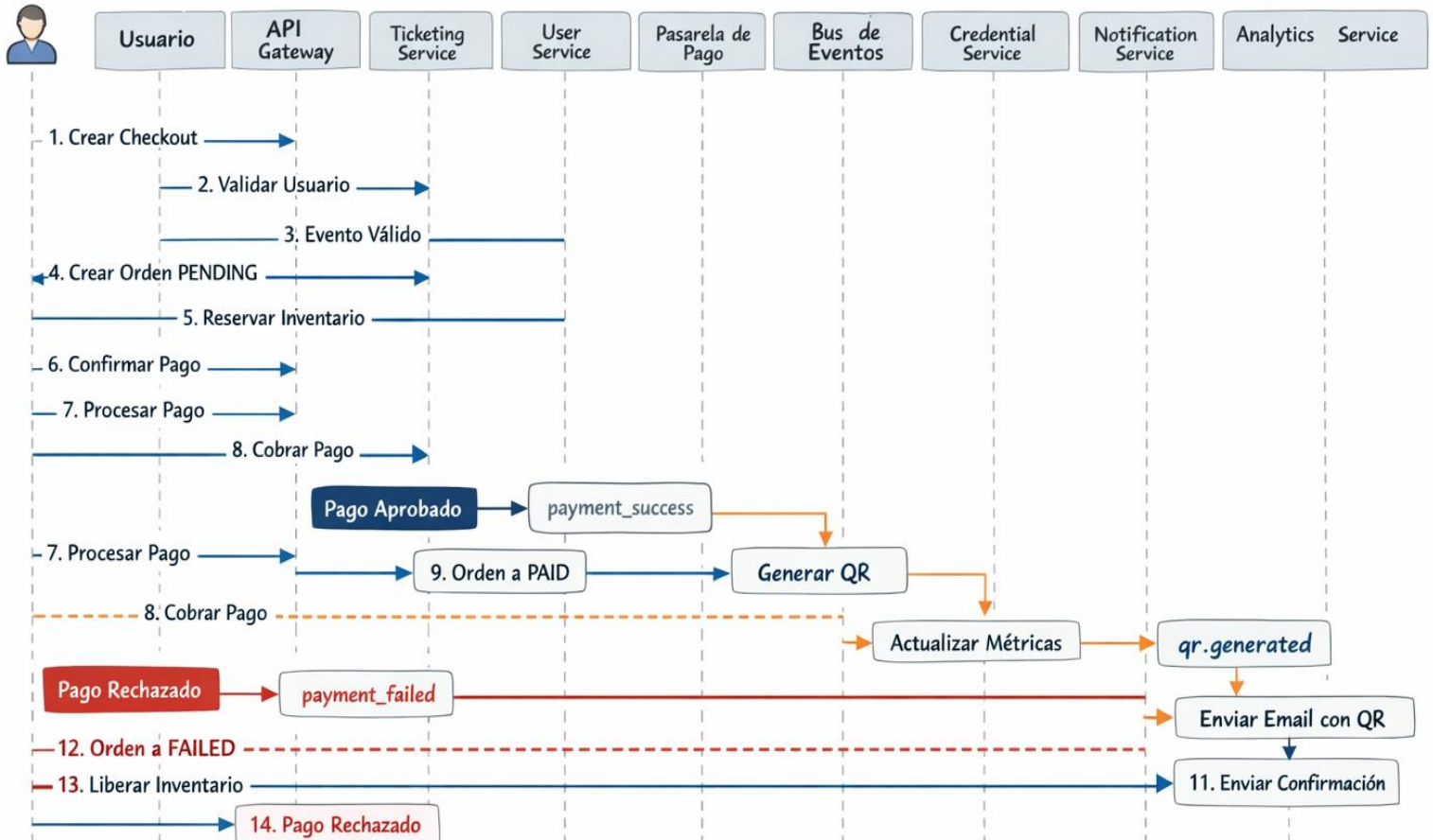
Beneficios

evita saturación de recursos, reduce tiempos de espera innecesarios, protege la estabilidad del sistema, permite degradación controlada.

Ejemplo en EventHive

Si la pasarela de pago comienza a fallar repetidamente, el circuit breaker se abre. A partir de ese momento, Ticketing deja de insistir con nuevas solicitudes de cobro durante una ventana corta y responde con un error controlado al usuario. Cuando pasa el tiempo de enfriamiento, el sistema prueba nuevamente algunas solicitudes en modo half-open.

DIAGRAMA DE SECUENCIA DE UN FLUJO COMPLEJO

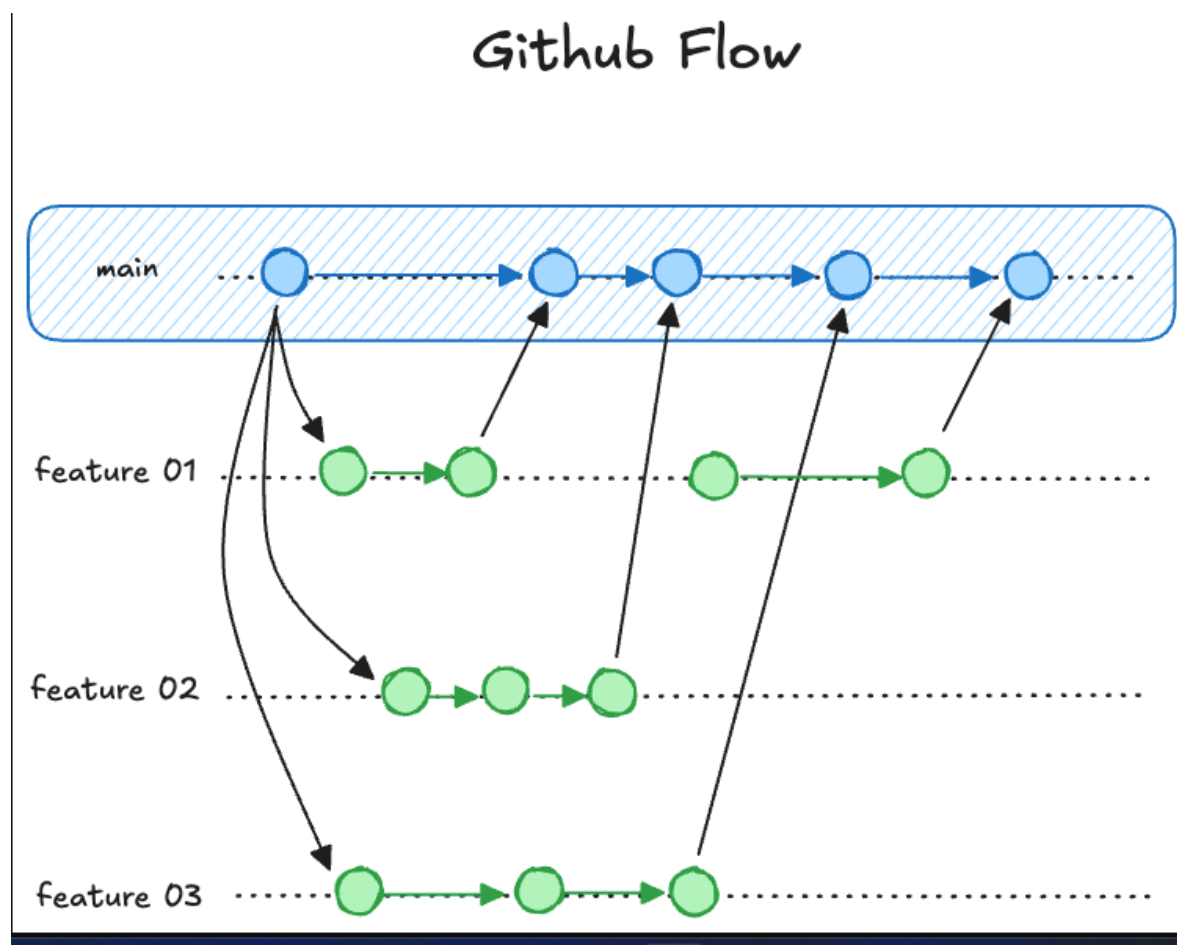


En este apartado, explico cómo planeo estructurar el desarrollo, las pruebas automáticas y la implementación de EventHive. El objetivo es mantener todo sencillo, veloz y seguro, dado que estamos trabajando con microservicios que requieren actualizaciones independientes, especialmente Ticketing cuando se producen picos en las ventas. Me gustaría que cualquier modificación se experimente y se despliegue sin necesidad de reconfigurar todo el sistema.

7. Estrategia DevOps

7.1 Proceso de trabajo en Git (GitHub Flow)

Figura 1: Diagrama del flujo GitHub Flow adaptado a EventHive.



Opté por GitHub Flow porque es el que más se acomoda a proyectos de este tipo, que implican entregas ininterrumpidas, microservicios autónomos y despliegues recurrentes en

Render y Vercel. Gitflow, con sus ramas de release y develop que crean más conflictos de fusión de los que son necesarios, me parecía excesivamente complicado.

Motivos principales para usar EventHive:

Solo una rama principal (main) que siempre está preparada para la producción.

Cada nueva modificación se realiza en una rama corta y, tras las pruebas y la revisión, se fusiona rápidamente.

Facilita que cada microservicio (o el frontend) se actualice de manera autónoma, sin necesidad de esperar a que lo haga el resto del equipo.

Cuando algo llega a main, Render y Vercel se encargan de desplegarlo automáticamente, lo cual simplifica enormemente las cosas.

Cómo va a funcionar paso a paso:

1. Todo empieza desde la rama main.
2. Para cualquier tarea (nueva funcionalidad, bug, refactor, actualización de paquetes): creo una rama desde main con nombre descriptivo, por ejemplo:

✚ feature/ticketing-cupon-descuento

✚ fix/credential-evitar-qr-duplicado

✚ chore/actualizar-nestjs

Hago commits pequeños y claros (usando Conventional Commits para que sea fácil generar changelogs después): Ejemplo: feat(ticketing): agregar lógica de cupones promocionales

Cuando termino: hago push, abro un Pull Request hacia main.

✚ Pido al menos un review (usaré CODEOWNERS para que GitHub sugiera quién debe revisar cada servicio).

✚ El pipeline CI corre automáticamente (lint, tests, build).

Si todo pasa, merge con **squash & merge** para mantener la historia de main limpia.

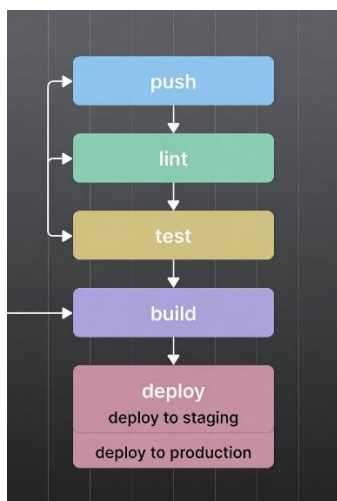
Al mergear, se elimina la rama y se dispara el despliegue automático.

Entornos que usaré:

- **Local:** docker-compose para probar PostgreSQL + Redis + todos los servicios juntos.
- **Preview:** Vercel genera previews automáticos por cada PR en el frontend. Para backend, si necesito validar antes, usaré un entorno temporal en Render.
- **Producción:** solo lo que está en main.

Estructura de repositorios: Prefiero un repo por microservicio (ej. eventhive-ticketing-service, eventhive-credential-service) + uno para el frontend, para que cada uno tenga su propio CI/CD y sea más independiente.

7.2 Pipeline CI/CD con GitHub Actions



(Muestra pasos: push → lint → test → build → deploy.)

Usaré GitHub Actions para automatizar pruebas y despliegue. Por cada PR y push a main correrá lo siguiente:

- Instalación de dependencias.
- Linting (ESLint + Prettier).
- Tests unitarios e integración (Jest o Vitest).
- Build (para verificar que compila bien, especialmente en NestJS).
- Cobertura de código (opcional, pero lo agregaré después).

Para Ticketing Service agregaré pruebas de carga básicas más adelante (con k6 o Artillery) para simular muchos usuarios comprando al mismo tiempo.

Cómo se dispara el despliegue: En Render desactivaré el auto-deploy y usaré **Deploy Hooks** (la forma más simple y recomendada). Cada servicio tendrá su propio hook URL guardado como secreto en GitHub. Cuando CI pase y sea merge a main, un curl simple lo activa.

Ejemplo de workflow para Ticketing Service (.github/workflows/ci-cd.yml):

YAML

name: CI/CD - Ticketing Service

on:

push:

branches: [main]

pull_request:

branches: [main]

jobs:

ci:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-node@v4

with:

node-version: '20'

cache: 'npm'

- run: npm ci
- run: npm run lint
- run: npm test -- --coverage
- run: npm run build # Para NestJS o si hay build step

deploy:

needs: ci

if: github.ref == 'refs/heads/main' && github.event_name == 'push'

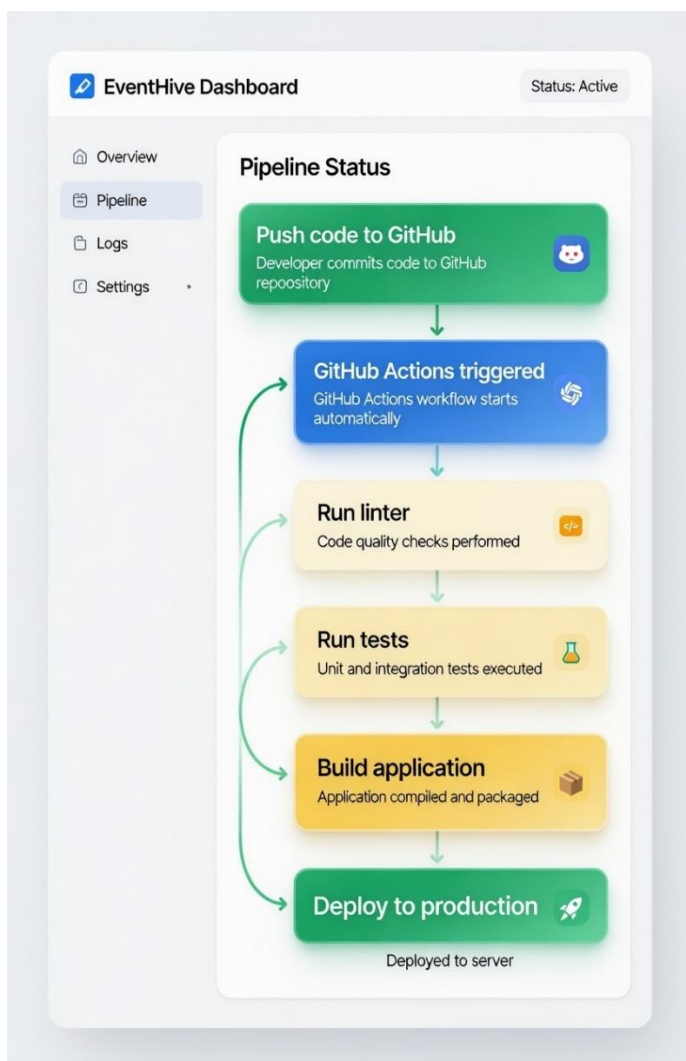
runs-on: ubuntu-latest

steps:

- name: Desplegar en Render

run: curl -X POST "\${{ secrets.RENDER_DEPLOY_HOOK_TICKETING }}"

Figura 2: Dashboard simulado del pipeline CI/CD de EventHive.



"Esta imagen representa de forma visual el flujo completo del pipeline CI/CD que implementaré en EventHive. Muestra cómo un push a GitHub activa automáticamente GitHub Actions, que ejecuta lint, tests, build y finalmente despliega a producción (Render para backend o Vercel para frontend). Cada paso tiene su icono y descripción para que sea fácil de entender el proceso automático que evita errores manuales y acelera los despliegues."

Cada microservicio tendrá su propio workflow y secreto (RENDER_DEPLOY_HOOK_TICKETING, etc.). El frontend en Vercel se maneja nativo con la integración de GitHub.

7.3 Estrategia de despliegue

- **Frontend (Next.js - panel organizadores y páginas públicas):** Vercel. Previews automáticos por PR y deploy en main. Ideal para SSR y SEO.
- **Backend (microservicios Node.js/NestJS):** Render como Web Services. Render build desde el repo Git con Docker y hace rolling updates (sin downtime).
- **Dockerfile básico que usaré (multi-stage para que sea liviano y rápido):**

Etapa 1: Build

FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build

Etapa 2: Producción

FROM node:20-alpine

WORKDIR /app

ENV NODE_ENV=production

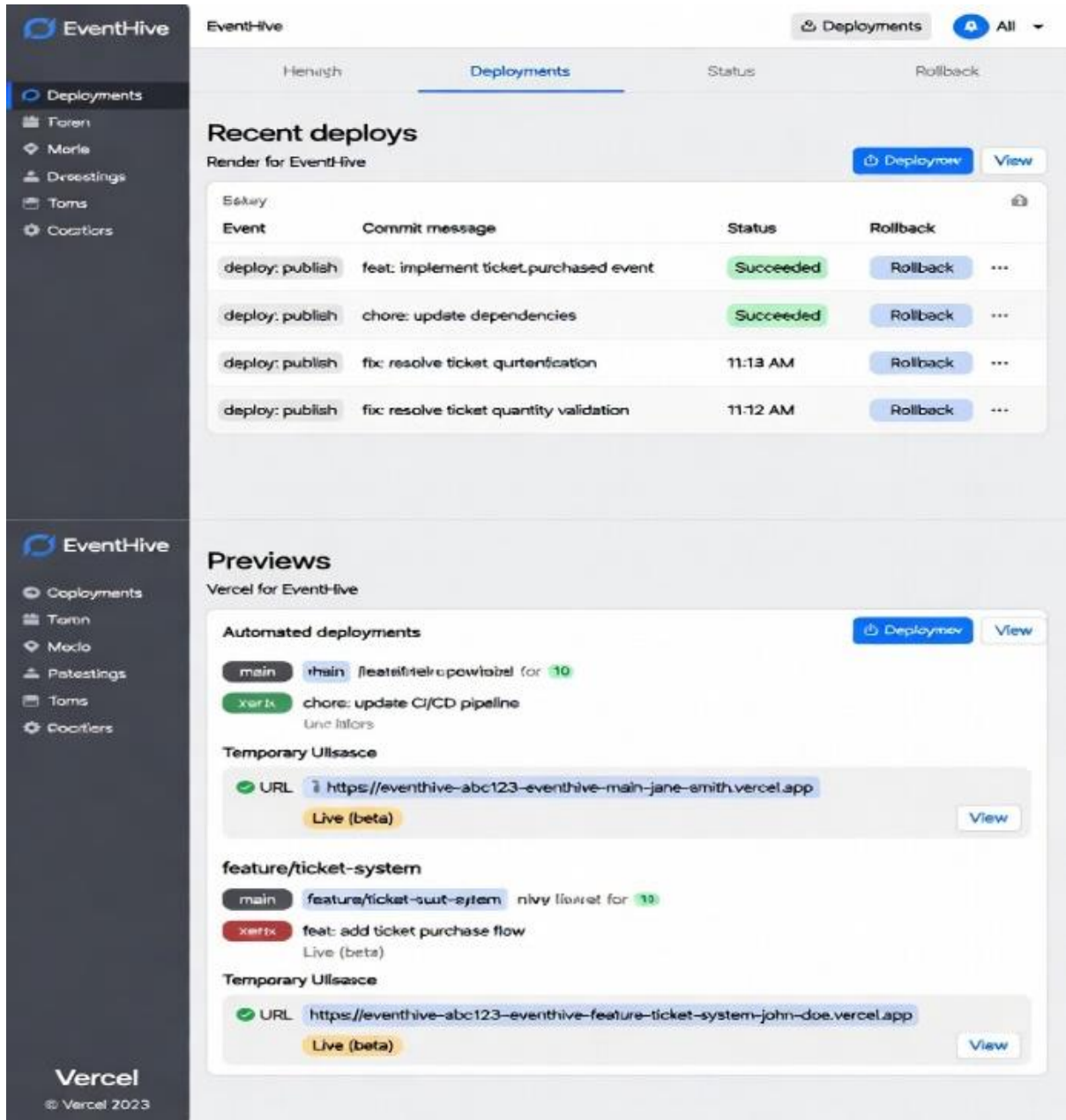
COPY --from=builder /app ./

EXPOSE 3000

CMD ["node", "dist/main.js"] # Ajustar según el entrypoint de NestJS

- **Migraciones de base de datos:** Las ejecutaré con prisma migrate deploy al inicio del container o como job manual al principio (para no arriesgar downtime).
- **Versionado de APIs:** Usaré rutas como /api/v1/events, /api/v1/tickets. Cuando lance v2 mantendré v1 por varios meses. Documentaré todo con Swagger (viene integrado en NestJS).

Figura 3: Historial de despliegues en Render (con ejemplos de commits).
Previews en Vercel (para probar cambios en frontend). Fuentes: Adaptadas de documentación oficial de GitHub, Render y Vercel.



The screenshot displays the EventHive dashboard, which is divided into two main sections: 'Recent deploys' and 'Previews'.

Recent deploys (Render for EventHive):

Event	Commit message	Status	Rollback
deploy: publish	feat: implement ticket purchased event	Succeeded	Rollback
deploy: publish	chore: update dependencies	Succeeded	Rollback
deploy: publish	fix: resolve ticket quantification	11:13 AM	Rollback
deploy: publish	fix: resolve ticket quantity validation	11:12 AM	Rollback

Previews (Vercel for EventHive):

Automated deployments:

- main** (selected): feat: implement ticket purchased event for 10
- next**: chore: update CI/CD pipeline

Temporary URL: <https://eventhive-abc123-eventhive-main-jane-smith.vercel.app> (Live (beta))

feature/ticket-system:

- main** (selected): feature/ticket-system nivy latest for 10
- next**: feat: add ticket purchase flow

Temporary URL: <https://eventhive-abc123-eventhive-feature-ticket-system-john-doe.vercel.app> (Live (beta))

The dashboard includes a sidebar with navigation links: Deployments, Events, Metrics, Deployments, Events, and Coauthors. The Vercel logo and copyright notice (© Vercel 2023) are visible at the bottom left.

Modelado de datos relacionales por microservicio

Cada microservicio de EventHive posee su propia base de datos PostgreSQL independiente alojada en Render, siguiendo el principio Database-per-Service, a continuación, únicamente las tablas, columnas, claves e índices.

User Service — Base de datos de usuarios

TABLA : users

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único del usuario
email	VARCHAR(255)	NOT NULL, UNIQUE	Correo electrónico — credencial de login
password_hash	VARCHAR(255)	NOT NULL	Hash de la contraseña (bcrypt)
full_name	VARCHAR(200)	NOT NULL	Nombre completo del usuario
role	VARCHAR(50)	NOT NULL	Rol del usuario (ATTENDEE, ORGANIZER, SPEAKER)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Fecha y hora de registro

Tablas: permissions y user_roles

Índices — User Service

Índice	Tabla	Columna	Tipo	Justificación
idx_users_email	users	email	UNIQUE B-Tree	Búsqueda rápida en login por email
idx_users_role	users	role	B-Tree	Filtrar usuarios por tipo de rol

User Service

La tabla users cumple la Tercera Forma Normal (3FN): cada columna depende únicamente de la clave primaria id y no existen dependencias transitivas entre columnas no clave.

El campo role almacena el rol directamente en la fila del usuario según el esquema definido. La tabla user_roles sugiere que el diseño contempla una relación N:M futura, pero su esquema no está definido aún.

Integridad sin FK entre servicios: los demás microservicios almacenan el id del usuario como referencia lógica. No existe FK física hacia esta base de datos desde otros servicios.

Base de datos de eventos

Tabla: events

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único del evento
organizer_id	UUID	NOT NULL	Ref. lógica al organizador (User Service)
name	VARCHAR(300)	NOT NULL	Nombre del evento
slug	VARCHAR(300)	NOT NULL, UNIQUE	URL amigable única del evento
description	TEXT	NULL	Descripción detallada del evento
start_date	TIMESTAMPTZ	NOT NULL	Fecha y hora de inicio
end_date	TIMESTAMPTZ	NOT NULL	Fecha y hora de cierre
location	TEXT	NULL	Ubicación física o URL virtual del evento
status	VARCHAR(20)	NOT NULL	Estado: DRAFT, PUBLISHED, CANCELLED, FINISHED

Tabla: speakers

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único del ponente
event_id	UUID	NOT NULL, FK → events(id)	Evento al que pertenece el ponente
name	VARCHAR(200)	NOT NULL	Nombre completo del ponente
bio	TEXT	NULL	Biografía del ponente
photo_url	TEXT	NULL	URL de la foto del ponente

Tabla: sessions

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único de la sesión
event_id	UUID	NOT NULL, FK → events(id)	Evento al que pertenece la sesión
speaker_id	UUID	NULL, FK → speakers(id)	Ponente asignado a la sesión
title	VARCHAR(400)	NOT NULL	Título de la sesión o charla

Índices — Event Service

Índice	Tabla	Columna	Tipo	Justificación
idx_events_slug	events	slug	UNIQUE B-Tree	Resolución de URL pública del evento
idx_events_organizer	events	organizer_id	B-Tree	Listar eventos de un organizador
idx_events_status	events	status	B-Tree	Filtrar eventos por estado (PUBLISHED)
idx_speakers_event	speakers	event_id	B-Tree	Listar ponentes de un evento
idx_sessions_event	sessions	event_id	B-Tree	Listar sesiones de un evento
idx_sessions_speaker	sessions	speaker_id	B-Tree	Sesiones asignadas a un ponente

EVENT SERVICE

Las tres tablas cumplen 3FN. Los ponentes y las sesiones se separan de la tabla events para evitar repetición de datos y permitir que una sesión tenga un solo ponente asignado.

La columna organizer_id en events es una referencia lógica al User Service. No existe FK física entre bases de datos. Si el User Service no está disponible, los datos del evento permanecen íntegros.

Consistencia eventual: cuando el Event Service publica el evento ticket.purchased, el Ticketing Service lee el event_id para operar de forma independiente sin necesidad de consultar constantemente al Event Service.

Base de datos de venta de entradas

Tabla: ticket_types

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único del tipo de entrada
event_id	UUID	NOT NULL	Ref. lógica al evento (Event Service)
name	VARCHAR(200)	NOT NULL	Nombre: General, VIP, Taller, etc.
price	NUMERIC(12,2)	NOT NULL, CHECK(price >= 0)	Precio unitario de la entrada
quantity_total	INTEGER	NOT NULL, CHECK(quantity_total > 0)	Stock total de entradas de este tipo
quantity_sold	INTEGER	NOT NULL, DEFAULT 0	Entradas vendidas — actualizado atómicamente
max_per_person	INTEGER	NOT NULL	Límite máximo de entradas por comprador

Tabla: carts

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único del carrito
user_id	UUID	NOT NULL	Ref. lógica al comprador (User Service)
status	VARCHAR(20)	NOT NULL	Estado: OPEN, RESERVED, EXPIRED, CONVERTED
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Fecha y hora de creación del carrito

Tabla: cart_items

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único del ítem
cart_id	UUID	NOT NULL, FK → carts(id)	Carrito al que pertenece el ítem
ticket_type_id	UUID	NOT NULL, FK → ticket_types(id)	Tipo de entrada seleccionada
quantity	INTEGER	NOT NULL, CHECK(quantity > 0)	Cantidad de entradas de este tipo

Tabla: orders

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único de la orden
user_id	UUID	NOT NULL	Ref. lógica al comprador (User Service)
event_id	UUID	NOT NULL	Ref. lógica al evento (Event Service)
total_amount	NUMERIC(12,2)	NOT NULL	Monto total de la compra
status	VARCHAR(20)	NOT NULL	Estado: PENDING, PAID, FAILED, REFUNDED
payment_intent_id	VARCHAR(255)	NULL, UNIQUE	ID de la transacción en la pasarela de pago

Tabla: order_items

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único del ítem
order_id	UUID	NOT NULL, FK → orders(id)	Orden a la que pertenece el ítem
ticket_type_id	UUID	NOT NULL, FK → ticket_types(id)	Tipo de entrada comprada
quantity	INTEGER	NOT NULL, CHECK(quantity > 0)	Cantidad comprada de este tipo
unit_price	NUMERIC(12,2)	NOT NULL	Precio unitario al momento de la compra

Índices — Ticketing Service

El documento de arquitectura define explícitamente dos índices críticos para este servicio:

Índice	Tabla	Columna	Tipo	Justificación (según doc. de arquitectura)
idx_ticket_types_event_id	ticket_types	event_id	B-Tree	Consultar tipos de entrada de un evento — definido como crítico en el documento
idx_orders_status	orders	status	B-Tree	Filtrar órdenes por estado de pago — definido como crítico en el documento
idx_carts_user_id	carts	user_id	B-Tree	Localizar carrito activo de un comprador
idx_cart_items_cart_id	cart_items	cart_id	B-Tree	Listar ítems de un carrito
idx_order_items_order_id	order_items	order_id	B-Tree	Listar ítems de una orden
idx_orders_payment_intent	orders	payment_intent_id	UNIQUE B-Tree	Evitar registrar el mismo pago dos veces

Ticketing Service

Las cinco tablas cumplen 3FN. Se destaca una desnormalización intencionada: el campo `unit_price` en `order_items` guarda el precio vigente al momento de la compra. Esto es necesario porque el precio en `ticket_types` puede cambiar con el tiempo y el historial de órdenes debe ser inmutable.

Prevención de sobreventa: la columna `quantity_sold` se actualiza con una operación atómica: `UPDATE ticket_types SET quantity_sold = quantity_sold + N WHERE id = ? AND (quantity_total - quantity_sold) >= N`. Si el `UPDATE` afecta 0 filas, la transacción se cancela, garantizando que no se vendan más entradas de las disponibles.

Consistencia eventual: al confirmar el pago, el Ticketing Service publica el evento `ticket.purchased`. El Credential Service y el Notification Service reaccionan a este evento de forma desacoplada.

Base de datos de credenciales

Tabla: credentials

Columna	Tipo inferido	Clave / Restricción	Descripción
id	UUID	PK, NOT NULL	Identificador único de la credencial
order_item_id	UUID	NOT NULL	Ref. lógica al ítem de orden (Ticketing Service)
ticket_type_id	UUID	NOT NULL	Ref. lógica al tipo de entrada
attendee_name	VARCHAR(200)	NOT NULL	Nombre del asistente al que se emite la entrada
qr_code_hash	VARCHAR(64)	NOT NULL, UNIQUE	Hash del QR para validación rápida por hash
is_used	BOOLEAN	NOT NULL, DEFAULT FALSE	Indica si la entrada ya fue escaneada en el check-in
used_at	TIMESTAMPTZ	NULL	Fecha y hora en que se realizó el check-in
used_by	UUID	NULL	ID del staff que realizó la validación

Índices — Credential Service

Índice	Tabla	Columna	Tipo	Justificación
idx_credentials_qr_hash	credentials	qr_code_hash	UNIQUE B-Tree	Validación instantánea del QR por hash — propósito central del campo según el documento
idx_credentials_order_item	credentials	order_item_id	B-Tree	Localizar credencial a partir de un ítem de orden
idx_credentials_is_used	credentials	is_used	B-Tree	Filtrar entradas pendientes de uso en check-in

Credential Service

La tabla cumple 3FN. La columna attendee_name desnormaliza el nombre del asistente para que el servicio pueda operar de forma autónoma durante el check-in sin necesidad de consultar al User Service en tiempo real, lo que garantiza disponibilidad máxima en los puntos de acceso.

El campo qr_code_hash almacena el hash del contenido del QR, no el QR completo, tal como lo indica el documento: "permite una validación rápida por hash en lugar de por el código completo". La búsqueda por UNIQUE B-Tree sobre este campo es $O(\log n)$.

Integridad sin FK: order_item_id y ticket_type_id son referencias lógicas hacia la base de datos del Ticketing Service. La credencial se genera de forma reactiva al evento ticket.purchased y no requiere FK física.

Base de datos de agenda

Tabla: favorites

Columna	Tipo inferido	Clave / Restricción	Descripción
user_id	UUID	PK (compuesta), NOT NULL	Ref. lógica al asistente (User Service)
session_id	UUID	PK (compuesta), NOT NULL	Ref. lógica a la sesión (Event Service)
created_at	TIMESTAMPZ	NOT NULL, DEFAULT NOW()	Fecha en que el asistente marcó la sesión como favorita

Tabla: ratings

Columna	Tipo inferido	Clave / Restricción	Descripción
session_id	UUID	PK (compuesta), NOT NULL	Ref. lógica a la sesión calificada (Event Service)
user_id	UUID	PK (compuesta), NOT NULL	Ref. lógica al asistente que califica (User Service)
score	SMALLINT	NOT NULL	Puntuación de la sesión
comment	TEXT	NULL	Comentario opcional del asistente sobre la sesión

Índices — Agenda Service

Índice	Tabla	Columna(s)	Tipo	Justificación
pk_favorites	favorites	user_id, session_id	UNIQUE B-Tree (PK)	Garantiza que un asistente no marque la misma sesión dos veces
idx_fav_user	favorites	user_id	B-Tree	Listar todas las sesiones favoritas de un asistente
idx_fav_session	favorites	session_id	B-Tree	Contar favoritos de una sesión
pk_ratings	ratings	session_id, user_id	UNIQUE B-Tree (PK)	Un asistente solo puede calificar cada sesión una vez
idx_ratings_session	ratings	session_id	B-Tree	Calcular promedio de calificaciones de una sesión

Agenda Service

Ambas tablas cumplen 3FN. Las claves primarias compuestas (user_id, session_id) en favorites y (session_id, user_id) en ratings eliminan la necesidad de un surrogate key y garantizan unicidad de forma natural.

Las columnas user_id y session_id son referencias lógicas hacia el User Service y el Event Service respectivamente. No existen FK físicas entre bases de datos.

Mobile API Service

Tablas definidas en Analytics DB

Tabla	Descripción funcional según documento de arquitectura	Esquema interno
daily_sales	Métricas de ventas diarias (ingresos, número de entradas vendidas)	No definido en el documento
session_attendance	Asistencia a sesiones: aforo en vivo durante el evento	No definido en el documento
attendee_demographics	Perfil demográfico de asistentes: ubicación, tipo de entrada, ingresos por categoría	No definido en el documento

Características del Analytics Service

Ejecuta procesos ETL periódicamente (cada pocos minutos) sobre réplicas de solo lectura de las bases de datos de otros servicios.

- ✚ No afecta el rendimiento de los servicios transaccionales al operar sobre réplicas.
- ✚ Expone dashboards para los organizadores con: ventas en tiempo real, asistencia a sesiones, perfil demográfico e ingresos por categoría.
- ✚ Consume el evento ticket.purchased del Ticketing Service para alimentar métricas de ventas en tiempo real.

Normalización: al tratarse de datos pre-agregados (OLAP) el modelo puede aplicar desnormalización intencional para optimizar consultas de dashboards, pero esto quedará definido cuando el equipo especifique el esquema interno de cada tabla.

Resumen de Esquemas por Microservicio

La siguiente tabla resume el estado de definición de cada base de datos según el proyecto:

Microservicio	Base de datos	Tablas definidas con columnas	Tablas sin esquema definido
User Service	User DB	users	permissions, user_roles
Event Service	Event DB	events, speakers, sessions	—
Ticketing Service	Ticketing DB	ticket_types, carts, cart_items, orders, order_items	—
Credential Service	Credential DB	credentials	—
Agenda Service	Agenda DB	favorites, ratings	—
Notification Service	No definida	—	Toda la BD pendiente de definición
Mobile API Service	Sin BD propia	—	BFF sin base de datos propia
Analytics Service	Analytics DB	—	daily_sales, session_attendance, attendee_demographics